TML26 - STOLEN MODEL DETECTION AS2

Sandhya Badiger
saba00012@stud.uni-saarland.de

Mohammad Shaique Solanki
moso00002@stud.uni-saarland.de

1 INTRODUCTION

The goal of this assignment is to detect which of the 360 suspect image classification models were stolen from a given target model. For each suspect model, we output a continuous stealing confidence score, where a higher score means that the model is more likely to be stolen or derived from the target model.

A model can be stolen not only by direct copying, but also by fine-tuning, post-processing, using intermediate checkpoints, or model extraction/distillation. Since the evaluation metric is TPR at 5% FPR, the main goal is to rank truly stolen models high while keeping false positives low [1, 2].

2 APPROACH

Our approach investigates the suspect models through two primary methods: parameter-space similarity to detect simple theft (e.g., direct weight copying or fine-tuning), and function-space similarity to detect behavioral mimicry (e.g., knowledge distillation) [3].

We utilized the provided CIFAR-style ResNet-18 architecture with a modified first convolution layer, no max-pooling, and a final fully connected layer for 100 classes. All evaluations were conducted using the provided `train_main_idx.json` indices to probe the models with the target model's memorized training distribution.

2.1 FINAL SUBMITTED METHOD (THE BASELINE)

Our highest-scoring public leaderboard submission (TPR@5%FPR $= 0.574074$) was achieved using a straightforward, unweighted combination of weight similarity and logit agreement on clean, unaugmented data **??**.

To evaluate parameter-space similarity, we flattened the final fully connected layer weights of both models and computed their cosine similarity, normalizing the $[-1, 1]$ output to a $[0, 1]$ confidence bounded score:

$$\text{weight\_score} = \frac{\cos(w_t, w_s) + 1}{2}$$

To evaluate function-space similarity, we passed the clean probe dataset through both models and calculated the Mean Squared Error (MSE) between their output logits. This distance was converted into a bounded similarity score:

$$\text{logit\_score} = \frac{1}{1 + \text{avg\_mse}}$$

The final stealing confidence score simply averaged these two metrics:

$$\text{final\_score} = 0.5 \cdot \text{weight\_score} + 0.5 \cdot \text{logit\_score}$$

Despite its simplicity, this method proved highly stable, effectively capturing direct copies while adequately penalizing models whose logit distributions diverged heavily from the target.

2.2 ADVANCED FUNCTION-SPACE PROBING (UNSUCCESSFUL ITERATIONS)

In an attempt to improve our baseline score, we hypothesized that the target model's highly specific training augmentations (specifically, a biased random crop with reflection padding) acted as an inherent watermark. We upgraded our scoring pipeline to take the maximum of the two scores rather

than the average, $\max(\text{weight\_score}, \text{behavior\_score})$, to prevent pure distilled models from being penalized by their orthogonal weights. Furthermore, we swapped MSE for Kullback-Leibler (KL) divergence with temperature scaling to better capture the shape of the "dark knowledge" in the logits [1].

Despite being conceptually stronger, these changes failed to improve our public leaderboard score. We theorize this is due to the "distillation gap." If an attacker extracted the target model using clean, standard data, their surrogate model never learned the target's specific invariants for biased cropping. When probed with aggressive augmentations, the stolen models' logits scrambled unpredictably, artificially lowering their similarity score and inducing false negatives. Clean data ultimately provided the sharpest signal.

2.3 Adversarial Fingerprinting & Resource Constraints

As a final approach, we attempted to utilize Adversarial Fingerprinting (Inference Attacks). By using the Fast Gradient Sign Method (FGSM) on the target model, we aimed to forge adversarial examples that act as an undeniable fingerprint, assuming stolen models would exhibit high adversarial transferability and make the exact same misclassifications [3].

Unfortunately, we were unable to fully realize this approach. Since Thursday of the assignment week, we experienced severe, persistent compute resource failures and job scheduling dropouts on our university HPC cluster. Due to these infrastructure limitations, we were unable to properly sweep for the optimal perturbation magnitude (ϵ) or reliably run the adversarial inference loop across all 360 models. Consequently, we reverted to our stable, clean-data baseline for the final evaluation.

3 Results

Our best public leaderboard result was:

TPR@5%FPR = 0.574074.

This result was obtained using final-layer cosine similarity and logit MSE agreement. The method worked well for models that either had similar classifier weights or produced similar logits on the probe dataset. The approach is simple and interpretable, but it is heuristic because the true stolen/not-stolen labels are hidden. Also, using only the final layer may miss models where the classifier head was changed, and the logit probe may not capture all heavily transformed stolen models.

4 Conclusion

Model stealing is a security concern because training strong models requires data, compute, time, and engineering effort. If an attacker copies or extracts such a model, they can benefit from this investment without permission and may also use the stolen model for further attacks.

In this assignment, we detected stolen models using both weight-level and output-level similarity. The final submitted method used final-layer cosine similarity and logit MSE agreement, which gave our best leaderboard score. Future improvements could include stronger adversarial fingerprints, more diverse probe inputs, and layer-wise similarity analysis.

The link to the code(s) of this work can be found here. **GitHub Repository:** `https://github.com/CruelMarco/Trustworthy-Machine-Learning`

References

[1] Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. High Accuracy and High Fidelity Extraction of Neural Networks. USENIX Security, 2020. `https://arxiv.org/abs/1909.01838`

[2] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff Nets: Stealing Functionality of Black-Box Models. CVPR, 2019. `https://arxiv.org/abs/1812.02766`

[3] Harshay Shah, Sung Min Park, Andrew Ilyas, and Aleksander Madry. ModelDiff: A Framework for Comparing Learning Algorithms. ICML, 2023. https://arxiv.org/abs/2211.12491